

HRMS Module 1: Authentication & Identity Management - Complete Technical Bible

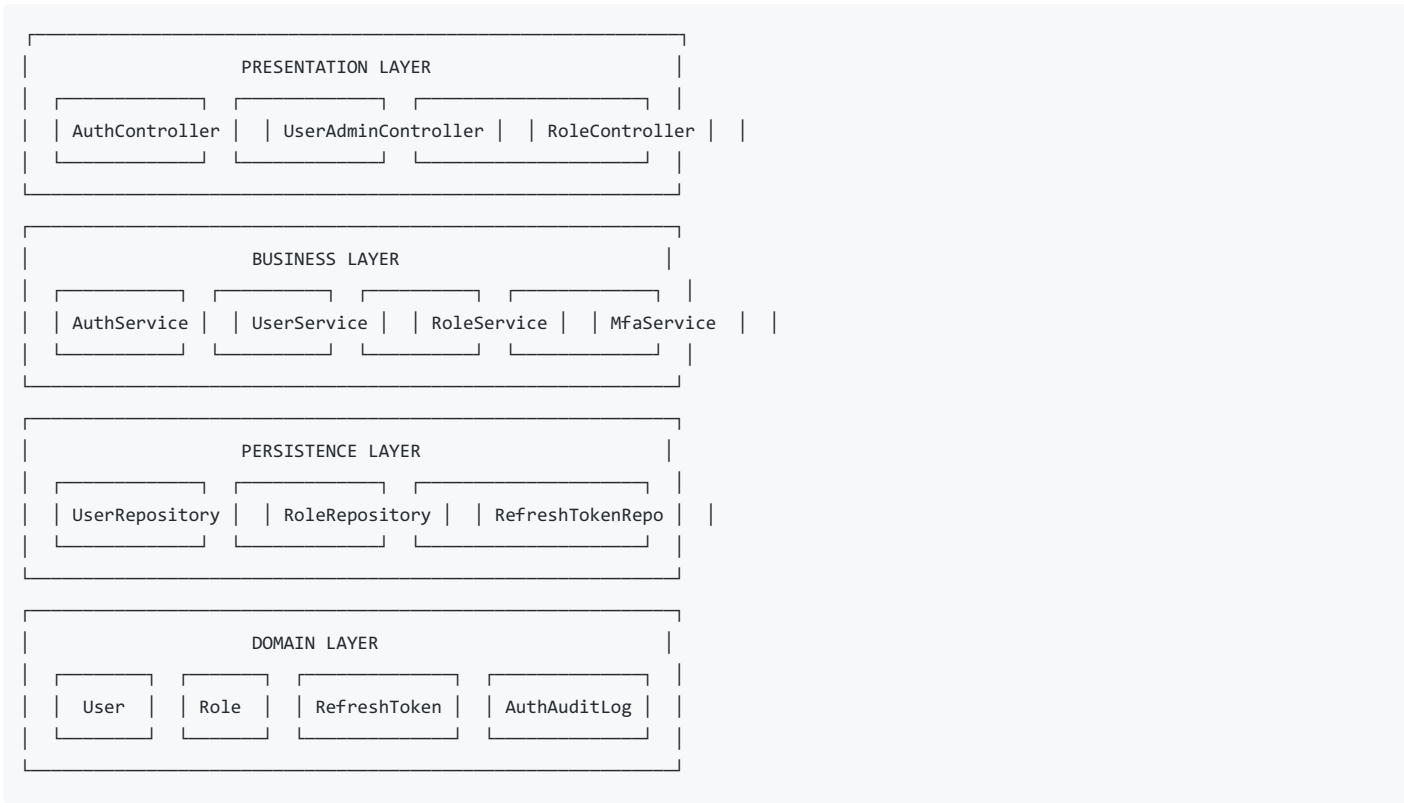
Table of Contents

- [Architecture Overview](#)
- [Database Design & Schema](#)
- [Entity Layer Deep Dive](#)
- [Repository Layer Analysis](#)
- [Service Layer Implementation](#)
- [Security Infrastructure](#)
- [Controller Layer & API Design](#)
- [Event-Driven Architecture](#)
- [Utility Components](#)
- [Configuration Management](#)
- [Complete Data Flow](#)
- [Security Considerations](#)

Architecture Overview

Microservice Structure

Module 1 follows a **strictly layered architecture** with clear separation of concerns:



Technology Stack

- Framework:** Spring Boot 3.x + Spring Security 6
- Database:** PostgreSQL with UUID primary keys
- Authentication:** JWT (JSON Web Tokens)
- Password Hashing:** Argon2 + BCrypt
- MFA:** TOTP (Time-based One-Time Password)
- API Documentation:** OpenAPI 3 (Ready for Swagger)
- Build Tool:** Maven/Gradle ready structure

Database Design & Schema

Core Tables & Relationships

1. Users Table - Central Identity Store

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  username VARCHAR(100) NOT NULL UNIQUE,          -- Case-insensitive login  
  email VARCHAR(150) NOT NULL UNIQUE,             -- Unique email constraint  
  password_hash VARCHAR(255),                     -- Nullable for SSO users  
  status user_status NOT NULL DEFAULT 'ACTIVE',    -- ACTIVE, DISABLED, LOCKED  
  employee_id UUID NULL,                           -- FK to Employee module  
  mfa_method mfa_method DEFAULT 'NONE',           -- TOTP, SMS, EMAIL, NONE  
  mfa_secret VARCHAR(255),                         -- Encrypted TOTP secret  
  failed_login_count INT DEFAULT 0,                -- Security tracking  
  last_failed_login_at TIMESTAMP,                  -- Security timestamp  
  last_login_at TIMESTAMP,                         -- Activity tracking  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Business Logic Implications:

- password_hash is **nullable** to support SSO-only users
- employee_id creates **loose coupling** with Employee module
- failed_login_count enables **account lockout** after threshold
- mfa_secret is **encrypted at application level** before storage

2. Roles Table - RBAC Foundation

```
CREATE TABLE roles (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  name role_name NOT NULL UNIQUE,  -- ENUM: ROLE_ADMIN, ROLE_HR, etc.  
  description TEXT,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Pre-defined Roles:

- ROLE_ADMIN - Full system access
- ROLE_HR - Human Resources management
- ROLE_MANAGER - Team management
- ROLE_EMPLOYEE - Basic user access
- ROLE_FINANCE - Financial data access
- ROLE_AUDITOR - Read-only audit access
- ROLE_SYSTEM - Service-to-service communication

3. User-Roles Junction Table - Many-to-Many with History

```
CREATE TABLE user_roles (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  role_id UUID NOT NULL REFERENCES roles(id) ON DELETE CASCADE,  
  assigned_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  revoked_at TIMESTAMP,                                -- Soft delete for audit  
  UNIQUE(user_id, role_id)                             -- Prevent duplicate assignments  
);
```

Audit Trail Design:

- assigned_at tracks when role was granted
- revoked_at enables **soft deletion** and audit history
- Unique constraint prevents **role duplication**

4. Refresh Tokens Table - Session Management

```
CREATE TABLE refresh_tokens (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  token_hash VARCHAR(255) NOT NULL,           -- Hashed token value  
  issued_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  expires_at TIMESTAMP NOT NULL,               -- Absolute expiry  
  revoked_at TIMESTAMP,                       -- Manual revocation  
  ip_address VARCHAR(45),                     -- IPv4/IPv6 support  
  user_agent TEXT                             -- Browser/device info  
);
```

Security Features:

- **Token Hashing:** Raw tokens never stored in database
- **IP Tracking:** Geographic and device session tracking
- **Cascade Delete:** Automatic cleanup when user is deleted

5. Auth Audit Log Table - Compliance & Security

```
CREATE TABLE auth_audit_log (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  actor_user_id UUID REFERENCES users(id),      -- Null for system actions  
  action VARCHAR(50) NOT NULL,                 -- LOGIN, LOGOUT, etc.  
  status VARCHAR(20),                          -- SUCCESS/FAILURE  
  ip_address VARCHAR(45),  
  meta JSONB,                                  -- Flexible metadata storage  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Audit Actions Tracked:

- LOGIN, LOGIN_FAILED, LOGOUT
- PASSWORD_RESET, TOKEN_REFRESH, TOKEN_REVOKE
- MFA_ENABLED, MFA_DISABLED

🔒 Entity Layer Deep Dive

User Entity - The Core Identity

```

@Entity
@Table(name = "users")
public class User {
    // Core Identification
    private UUID id;
    private String username;    // Login identifier
    private String email;      // Communication channel

    // Security Attributes
    private String passwordHash; // Nullable for SSO
    private UserStatus status;   // ACTIVE, DISABLED, LOCKED
    private MfaMethod mfaMethod; // Multi-factor method
    private String mfaSecret;    // Encrypted secret

    // Security Tracking
    private int failedLoginCount;
    private Instant lastFailedLoginAt;
    private Instant lastLoginAt;

    // Integration
    private UUID employeeId;    // Cross-module reference

    // Timestamps
    private Instant createdAt;
    private Instant updatedAt;
}

```

Business Logic Methods:

1. Password Validation

```

public boolean checkPassword(String plainPassword) {
    // Delegates to PasswordEncoderUtil for secure comparison
    return passwordEncoder.matches(plainPassword, this.passwordHash);
}

```

Security Flow:

1. **Input:** Plain text password from login request
2. **Processing:** Argon2/BCrypt comparison with stored hash
3. **Output:** Boolean result without exposing timing attacks

2. Failed Login Tracking

```

public void incrementFailedLogin() {
    this.failedLoginCount++;
    this.lastFailedLoginAt = Instant.now();

    // Auto-lock after threshold (configurable)
    if (this.failedLoginCount >= MAX_FAILED_ATTEMPTS) {
        this.lock();
    }
}

```

Security Logic:

- **In-memory counter** for performance
- **Automatic lockout** after configurable threshold
- **Timestamp tracking** for security monitoring

3. MFA Management

```
public void setMfa(MfaMethod method, String secret) {
    this.mfaMethod = method;
    this.mfaSecret = encryptSecret(secret); // Application-level encryption
}
```

Encryption Strategy:

- **TOTP secrets** encrypted before database storage
- **Encryption key** managed separately from database
- **No plaintext secrets** in logs or memory dumps

RefreshToken Entity - Session Persistence

```
@Entity
@Table(name = "refresh_tokens")
public class RefreshToken {
    private UUID id;
    private UUID userId;           // Foreign key to user
    private String tokenHash;      // SHA-256 hashed token
    private Instant issuedAt;
    private Instant expiresAt;
    private Instant revokedAt;     // Nullable for active tokens
    private String ipAddress;
    private String userAgent;
}
```

Token Validation Logic:

```
public boolean isExpired() {
    return Instant.now().isAfter(this.expiresAt);
}

public boolean isRevoked() {
    return this.revokedAt != null;
}

public boolean isValid() {
    return !isExpired() && !isRevoked();
}
```

Security Features:

- **Token Rotation:** New refresh token on each use
- **Hash Storage:** Prevents database compromise from token theft
- **Revocation Tracking:** Manual and automatic token invalidation

📦 Repository Layer Analysis

UserRepository - Identity Queries

```
public interface UserRepository extends JpaRepository<User, UUID> {

    // Authentication Queries
    Optional<User> findByUsernameIgnoreCase(String username);
    Optional<User> findByEmailIgnoreCase(String email);

    // Validation Queries
    boolean existsByUsername(String username);
    boolean existsByEmail(String email);

    // Administrative Queries
    Page<User> findAll(Pageable pageable);
}
```

Query Optimization:

- **Case-insensitive searches** for username/email
- **Boolean exists checks** for validation performance
- **Pagination support** for large user bases

RefreshTokenRepository - Session Management

```
public interface RefreshTokenRepository extends JpaRepository<RefreshToken, UUID> {  
  
    // Token Validation  
    Optional<RefreshToken> findByTokenHash(String tokenHash);  
  
    // User Session Management  
    List<RefreshToken> findByUserIdAndRevokedAtIsNull(UUID userId);  
  
    // Bulk Operations  
    @Modifying  
    @Query("UPDATE RefreshToken rt SET rt.revokedAt = CURRENT_TIMESTAMP WHERE rt.userId = :userId AND rt.revokedAt IS NULL")  
    void revokeAllByUserId(@Param("userId") UUID userId);  
}
```

Session Management Features:

- **Find active tokens** for user session listing
- **Bulk revocation** for password changes or security events
- **Token hash lookup** for refresh token validation

Custom Query Patterns

Soft Delete with JPA

```
// In UserRoleRepository  
@Modifying  
@Query("UPDATE UserRole ur SET ur.revokedAt = CURRENT_TIMESTAMP WHERE ur.userId = :userId AND ur.roleId = :roleId AND ur.revokedAt IS NULL")  
void softDeleteByUserIdAndRoleId(@Param("userId") UUID userId, @Param("roleId") UUID roleId);
```

Advantages:

- **Audit trail** maintained indefinitely
- **Data integrity** with soft deletion
- **Historical reporting** capabilities

⚙️ Service Layer Implementation

AuthService - Core Authentication Logic

Authentication Flow

```

public AuthResponse authenticate(AuthRequest request) {
    // 1. User Lookup
    User user = userRepository.findByUsernameIgnoreCase(request.getUsername())
        .orElseThrow(() -> new AuthenticationException("Invalid credentials"));

    // 2. Account Status Check
    if (user.getStatus() != UserStatus.ACTIVE) {
        auditService.recordLoginFailure(...);
        throw new AccountLockedException("Account " + user.getStatus());
    }

    // 3. Password Validation
    if (!passwordEncoder.matches(request.getPassword(), user.getPasswordHash())) {
        user.incrementFailedLogin();
        userRepository.save(user);
        throw new AuthenticationException("Invalid credentials");
    }

    // 4. MFA Validation (if enabled)
    if (user.getMfaMethod() != MfaMethod.NONE) {
        validateMfaCode(user, request.getMfaCode());
    }

    // 5. Success Processing
    user.resetFailedLogin();
    user.setLastLoginAt(Instant.now());
    userRepository.save(user);

    // 6. Token Generation
    String accessToken = tokenService.createAccessToken(...);
    RefreshTokenDto refreshToken = tokenService.createRefreshToken(...);

    // 7. Audit Logging
    auditService.recordLoginSuccess(...);

    return buildAuthResponse(accessToken, refreshToken, user);
}

```

Token Refresh with Rotation

```

public AuthResponse refreshToken(String refreshToken) {
    // 1. Token Validation
    String tokenHash = TokenHashUtil.hashToken(refreshToken);
    RefreshToken storedToken = refreshTokenRepository.findByTokenHash(tokenHash)
        .orElseThrow(() -> new TokenException("Invalid refresh token"));

    // 2. Security Checks
    if (storedToken.isExpired() || storedToken.isRevoked()) {
        throw new TokenException("Refresh token expired or revoked");
    }

    // 3. User Validation
    User user = userRepository.findById(storedToken.getUserId())
        .orElseThrow(() -> new UserNotFoundException("User not found"));

    // 4. Token Rotation (Security Best Practice)
    storedToken.revoke(); // Revoke old token
    refreshTokenRepository.save(storedToken);

    // 5. Issue New Tokens
    String newAccessToken = tokenService.createAccessToken(...);
    RefreshTokenDto newRefreshToken = tokenService.createRefreshToken(...);

    // 6. Audit Trail
    auditService.recordTokenRefresh(...);

    return buildAuthResponse(newAccessToken, newRefreshToken, user);
}

```

Security Benefits of Token Rotation:

- **Prevents replay attacks** - old tokens become invalid
- **Shortens exposure window** - compromised tokens have limited lifespan
- **Enables detection** - reused tokens indicate security breach

MfaService - Multi-Factor Authentication

TOTP Provisioning Flow

```

public MfaProvisioningDto provisionMfa(UUID userId, MfaMethod method) {
    User user = userRepository.findById(userId)
        .orElseThrow(() -> new UserNotFoundException("User not found"));

    // Generate TOTP Secret
    String secret = mfaUtil.generateSecret();

    // Create QR Code URI
    String provisioningUri = mfaUtil.getQrCodeImageUri(
        secret,
        user.getUsername(),
        "HRMS-Suite"
    );

    // Store encrypted secret (but don't enable yet)
    user.setMfaSecret(encryptionService.encrypt(secret));
    userRepository.save(user);

    return MfaProvisioningDto.builder()
        .method(method)
        .provisioningUri(provisioningUri)
        .secret(secret) // One-time display for user
        .build();
}

```

MFA Enablement with Verification


```

public void enableMfa(UUID userId, MfaEnableRequest request) {
    User user = userRepository.findById(userId)
        .orElseThrow(() -> new UserNotFoundException("User not found"));

    // Verify the provided code matches expected
    String decryptedSecret = encryptionService.decrypt(user.getMfaSecret());
    if (!mfaUtil.verifyCode(request.getVerificationCode(), decryptedSecret)) {
        throw new MfaValidationException("Invalid verification code");
    }

    // Only enable after successful verification
    user.setMfaMethod(request.getMethod());
    userRepository.save(user);
}

```

TOTP Security Implementation:

- **RFC 6238 compliant** - Standard time-based algorithm
- **30-second windows** - Balance security and usability
- **QR code generation** - Easy mobile app integration
- **Secret encryption** - Database-level protection

🔒 Security Infrastructure

JWT Token Provider

Token Generation

```

public String generateToken(String subject, Instant expiration, String jti) {
    return Jwts.builder()
        .setSubject(subject)                // User ID
        .setIssuer(jwtConfig.getIssuer())    // HRMS-Suite
        .setIssuedAt(Date.from(Instant.now())) // Issued time
        .setExpiration(Date.from(expiration)) // Expiry time
        .setId(jti)                          // Unique token ID
        .claim("preferred_username", username) // Human-readable ID
        .claim("roles", roles)                // Authorization claims
        .claim("employeeId", employeeId)      // Cross-module reference
        .signWith(getSecretKey())             // HMAC-SHA signing
        .compact();
}

```

JWT Claims Structure

```

{
  "sub": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "iss": "hrms-suite",
  "iat": 1672531200,
  "exp": 1672532100,
  "jti": "unique-token-id-12345",
  "preferred_username": "john.doe",
  "roles": ["ROLE_EMPLOYEE", "ROLE_MANAGER"],
  "employeeId": "b2c3d4e5-f6g7-8901-bcde-f23456789012"
}

```

Claim Purposes:

- `sub` - Subject (User ID) for identity
- `roles` - Authorization for method security
- `employeeId` - Cross-module data correlation
- `jti` - Unique identifier for token revocation

JWT Authentication Filter

Request Processing Flow

```
protected void doFilterInternal(HttpServletRequest request,
                                HttpServletResponse response,
                                FilterChain filterChain) {

    // 1. Extract Authorization Header
    String authHeader = request.getHeader("Authorization");
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        filterChain.doFilter(request, response);
        return;
    }

    // 2. Extract and Validate Token
    String token = authHeader.substring(7);
    if (jwtTokenProvider.validateToken(token)) {

        // 3. Parse Claims and Create Authentication
        TokenClaims claims = tokenService.parseAndValidateAccessToken(token);
        List<SimpleGrantedAuthority> authorities = extractAuthorities(claims);

        // 4. Set Security Context
        UsernamePasswordAuthenticationToken authentication =
            new UsernamePasswordAuthenticationToken(claims.getSub(), null, authorities);
        authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

        SecurityContextHolder.getContext().setAuthentication(authentication);
    }

    filterChain.doFilter(request, response);
}
```

Security Context Propagation:

- **Thread-local storage** of authentication details
- **Automatic cleanup** after request processing
- **Method-level security** integration via `@PreAuthorize`

Password Security

Argon2 Password Encoding

```
@Component
public class PasswordEncoderUtil {
    private final PasswordEncoder passwordEncoder;

    public PasswordEncoderUtil() {
        this.passwordEncoder = Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8();
    }

    public String encode(String rawPassword) {
        return passwordEncoder.encode(rawPassword);
    }

    public boolean matches(String rawPassword, String encodedPassword) {
        return passwordEncoder.matches(rawPassword, encodedPassword);
    }
}
```

Argon2 Advantages:

- **Memory-hard function** - Resistant to GPU/ASIC attacks
- **Time and memory configurable** - Adaptive to hardware
- **Salt automatically handled** - No manual salt management

🔒 Controller Layer & API Design

REST API Architecture

API Versioning & Base Structure

```
@RestController
@RequestMapping("/api/v1/auth") // Versioned API endpoint
public class AuthController {
    // All endpoints return standardized responses
    // Consistent error handling across all operations
}
```

Authentication Endpoints

1. Login Endpoint - Public Access

```
@PostMapping("/login")
public ResponseEntity<AuthResponse> login(@Valid @RequestBody AuthRequest request) {
    // Input Validation: @Valid triggers Bean Validation
    // Business Logic: AuthService.authenticate()
    // Response: Standardized AuthResponse DTO
    return ResponseEntity.ok(authService.authenticate(request));
}
```

Request Flow:

```
HTTP POST /api/v1/auth/login
Content-Type: application/json

{
  "username": "john.doe",
  "password": "securePassword123",
  "mfaCode": "123456" // Optional, required if MFA enabled
}
```

```
↓ Spring Validation (@Valid)
↓ AuthService.authenticate()
↓ User lookup + Password verification
↓ MFA validation (if enabled)
↓ Token generation
↓ Audit logging
```

```
HTTP 200 OK
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
  "expiresIn": "PT15M",
  "refreshToken": "dG9rZW4tcmlVcmVzaC1pZA...",
  "issuedAt": "2023-12-01T10:30:00Z",
  "user": {
    "id": "uuid",
    "username": "john.doe",
    "email": "john.doe@company.com",
    "status": "ACTIVE",
    "roles": ["ROLE_EMPLOYEE"],
    "employeeId": "employee-uuid"
  }
}
```

2. Token Refresh Endpoint - Public Access

```
@PostMapping("/refresh")
public ResponseEntity<AuthResponse> refreshToken(@RequestBody RefreshTokenRequest request) {
    return ResponseEntity.ok(authService.refreshToken(request.getRefreshToken()));
}
```

Security Implementation:

- **Token Rotation:** New refresh token issued, old one revoked
- **IP Tracking:** Maintains original request context
- **Audit Trail:** Records token refresh events

3. Logout Endpoint - Authenticated Access

```
@PostMapping("/logout")
public ResponseEntity<Void> logout(@RequestBody LogoutRequest request) {
    authService.revokeRefreshToken(request.getRefreshToken());
    return ResponseEntity.noContent().build(); // HTTP 204
}
```

Session Cleanup Process:

1. **Token Revocation:** Marks refresh token as revoked
2. **Cascade Effect:** Access token remains valid until expiry
3. **Audit Logging:** Records logout event for security monitoring

Administrative Endpoints

User Management Controller

```
@RestController
@RequestMapping("/api/v1/users")
public class UserAdminController {

    @PostMapping
    @PreAuthorize("hasRole('ROLE_ADMIN') or hasRole('ROLE_HR')")
    public ResponseEntity<UserDto> createUser(@Valid @RequestBody UserCreateRequest request) {
        return ResponseEntity.ok(userService.createUser(request));
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasRole('ROLE_ADMIN') or hasRole('ROLE_HR') or hasRole('ROLE_MANAGER') or @userSecurity.isSelf(#id)")
    public ResponseEntity<UserDto> getUser(@PathVariable UUID id) {
        return ResponseEntity.ok(userService.getUser(id));
    }
}
```

Role-Based Access Control Matrix:

Endpoint	ROLE_ADMIN	ROLE_HR	ROLE_MANAGER	ROLE_EMPLOYEE
POST /users	⌘	⌘	⌘	⌘
GET /users/{id}	⌘	⌘	Limited	Self-only
PUT /users/{id}	⌘	⌘	⌘	⌘
DELETE /users/{id}	⌘	⌘	⌘	⌘
POST /users/{id}/roles	⌘	⌘	⌘	⌘

Method Security Implementation

```
@Component("userSecurity")
public class UserSecurity {

    public boolean isSelf(UUID userId) {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
        String currentUserId = authentication.getName(); // JWT subject (user ID)
        return userId.toString().equals(currentUserId);
    }
}
```

Usage in Controllers:

```
@PreAuthorize("hasRole('ROLE_MANAGER') or @userSecurity.isSelf(#id)")
```

This enables:

- **Managers** to view their team members
- **Employees** to view only their own data
- **HR/Admin** to view all users

Password Reset Flow

Initiate Reset - Public Endpoint

```
@PostMapping("/password-reset/initiate")
public ResponseEntity<Void> initiatePasswordReset(@RequestBody PasswordResetInitiateRequest request) {
    authService.initiatePasswordReset(request.getEmail());
    return ResponseEntity.accepted().build(); // HTTP 202 - Async processing
}
```

Security Considerations:

- **No user enumeration:** Same response for valid/invalid emails
- **Rate limiting:** Prevents email bombing attacks
- **Secure tokens:** Time-limited, single-use reset tokens

Complete Reset - Public Endpoint

```
@PostMapping("/password-reset/complete")
public ResponseEntity<Void> completePasswordReset(@Valid @RequestBody PasswordResetCompleteRequest request) {
    authService.completePasswordReset(request.getToken(), request.getNewPassword());
    return ResponseEntity.noContent().build();
}
```

Password Policy Enforcement:

- **Complexity requirements:** Minimum length, character variety
- **History prevention:** Cannot reuse recent passwords
- **Session revocation:** All active sessions terminated after password change

MFA Management Endpoints

MFA Provisioning

```
@PostMapping("/mfa/provision")
public ResponseEntity<MfaProvisioningDto> provisionMfa(@RequestParam MfaMethod method) {
    UUID currentUserId = getCurrentUserId(); // From security context
    return ResponseEntity.ok(mfaService.provisionMfa(currentUserId, method));
}
```

Response Structure:

```
{
  "method": "TOTP",
  "provisioningUri": "otpauth://totp/HRMS-Suite:john.doe?secret=JBSWY3DPEHPK3PXP&issuer=HRMS-Suite",
  "secret": "JBSWY3DPEHPK3PXP"
}
```

MFA Enablement

```
@PostMapping("/mfa/enable")
public ResponseEntity<Void> enableMfa(@Valid @RequestBody MfaEnableRequest request) {
    UUID currentUserId = getCurrentUserId();
    mfaService.enableMfa(currentUserId, request);
    return ResponseEntity.noContent().build();
}
```

Verification Process:

1. **QR Code Scan:** User scans with authenticator app
2. **Code Verification:** User enters generated code
3. **Secret Storage:** Encrypted secret stored upon successful verification

📄 Event-Driven Architecture

Domain Events Design

User Lifecycle Events

```
// User Created Event
@Getter
public class UserCreatedEvent extends ApplicationEvent {
    private final UUID userId;
    public UserCreatedEvent(Object source, UUID userId) {
        super(source);
        this.userId = userId;
    }
}

// User Updated Event
@Getter
public class UserUpdatedEvent extends ApplicationEvent {
    private final UUID userId;
    public UserUpdatedEvent(Object source, UUID userId) {
        super(source);
        this.userId = userId;
    }
}

// User Disabled Event
@Getter
public class UserDisabledEvent extends ApplicationEvent {
    private final UUID userId;
    public UserDisabledEvent(Object source, UUID userId) {
        super(source);
        this.userId = userId;
    }
}
```

Security Events

```

// Refresh Token Revoked Event
@Getter
public class RefreshTokenRevokedEvent extends ApplicationEvent {
    private final UUID refreshTokenId;
    private final UUID userId;

    public RefreshTokenRevokedEvent(Object source, UUID refreshTokenId, UUID userId) {
        super(source);
        this.refreshTokenId = refreshTokenId;
        this.userId = userId;
    }
}

// Password Reset Requested Event
@Getter
public class PasswordResetRequestedEvent extends ApplicationEvent {
    private final String email;
    private final UUID userId; // Null if user not found (security measure)

    public PasswordResetRequestedEvent(Object source, String email, UUID userId) {
        super(source);
        this.email = email;
        this.userId = userId;
    }
}

```

Event Publishing Patterns

Service-Level Event Publishing

```

@Service
public class UserServiceImpl implements UserService {

    private final ApplicationEventPublisher eventPublisher;

    @Override
    @Transactional
    public UserDto createUser(UserCreateRequest request) {
        User user = // ... creation logic
        User savedUser = userRepository.save(user);

        // Publish after successful persistence
        eventPublisher.publishEvent(new UserCreatedEvent(this, savedUser.getId()));

        return convertToDto(savedUser);
    }
}

```

Transaction Boundary Considerations:

- **After commit:** Events published only after successful transaction
- **Synchronous processing:** Events processed in same thread by default
- **Error handling:** Failed event listeners don't roll back transaction

Event Consumers & Integration Points

Notification Module Integration

```

@Component
public class UserCreatedEventListener {

    private final NotificationService notificationService;

    @EventListener
    @Async // Process asynchronously to not block main flow
    public void handleUserCreated(UserCreatedEvent event) {
        // Send welcome email
        NotificationRequest welcomeEmail = buildWelcomeEmail(event.getUserId());
        notificationService.sendNotification(welcomeEmail);
    }

    @EventListener
    @Async
    public void handlePasswordReset>PasswordResetRequestedEvent event) {
        // Send password reset email
        NotificationRequest resetEmail = buildResetEmail(event.getEmail());
        notificationService.sendNotification(resetEmail);
    }
}

```

Audit Module Integration

```

@Component
public class SecurityEventListener {

    private final AuditService auditService;

    @EventListener
    public void handleTokenRevocation(RefreshTokenRevokedEvent event) {
        // Log token revocation for security audit
        auditService.logSecurityEvent(
            event.getUserId(),
            "REFRESH_TOKEN_REVOKED",
            Map.of("refreshTokenId", event.getRefreshTokenId())
        );
    }
}

```

Employee Module Integration

```

@Component
public class UserEventListener {

    private final EmployeeService employeeService;

    @EventListener
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void handleUserDisabled(UserDisabledEvent event) {
        // Disable corresponding employee record
        employeeService.disableEmployeeByUserId(event.getUserId());
    }
}

```

Integration Benefits:

- **Loose coupling:** Modules don't directly depend on each other
- **Eventual consistency:** Systems converge state asynchronously
- **Resilience:** Failure in one module doesn't block others

🔧 Utility Components

JWT Utility - Token Management

Claims Extraction & Validation

```
@Component
public class JwtUtil {

    public TokenClaims extractTokenClaims(Claims claims) {
        TokenClaims tokenClaims = new TokenClaims();

        // Extract standard claims
        tokenClaims.setSub(UUID.fromString(claims.getSubject()));
        tokenClaims.setPreferredUsername(claims.get("preferred_username", String.class));

        // Extract custom claims
        List<String> rolesList = claims.get("roles", List.class);
        if (rolesList != null) {
            List<RoleName> roles = rolesList.stream()
                .map(role -> RoleName.valueOf(role))
                .collect(Collectors.toList());
            tokenClaims.setRoles(roles);
        }

        // Extract timestamps
        tokenClaims.setIat(claims.getIssuedAt().toInstant());
        tokenClaims.setExp(claims.getExpiration().toInstant());
        tokenClaims.setJti(claims.getId());

        return tokenClaims;
    }
}
```

Claims Processing Logic:

- **Type safety:** UUID conversion with validation
- **Null safety:** Safe extraction of optional claims
- **Enum mapping:** String to RoleName enum conversion

Token Hash Utility - Secure Storage

Cryptographic Hashing

```
@UtilityClass
public class TokenHashUtil {

    public static String hashToken(String token) {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hash = digest.digest(token.getBytes(StandardCharsets.UTF_8));
            return Base64.getEncoder().encodeToString(hash);
        } catch (NoSuchAlgorithmException e) {
            throw new RuntimeException("SHA-256 algorithm not available", e);
        }
    }

    public static boolean verifyToken(String token, String hashedToken) {
        return hashToken(token).equals(hashedToken);
    }
}
```

Security Design:

- **SHA-256:** Cryptographically secure hashing algorithm
- **Base64 encoding:** Database-friendly string representation
- **Deterministic:** Same input always produces same output for verification

MFA Utility - TOTP Implementation

TOTP Code Generation & Verification

```
@Component
@Slf4j
public class MfaUtil {

    public String generateSecret() {
        return new DefaultSecretGenerator().generate(); // 32-character base32
    }

    public String getQrCodeImageUri(String secret, String username, String issuer) {
        QrData data = new QrData.Builder()
            .label(username)
            .secret(secret)
            .issuer(issuer)
            .algorithm(HashingAlgorithm.SHA1)
            .digits(6)
            .period(30) // 30-second windows
            .build();

        QrGenerator generator = new ZxingPngQrGenerator();
        byte[] imageData = generator.generate(data);

        return getDataUriForImage(imageData, generator.getImageMimeType());
    }

    public boolean verifyCode(String code, String secret) {
        TimeProvider timeProvider = new SystemTimeProvider();
        CodeGenerator codeGenerator = new DefaultCodeGenerator();
        CodeVerifier verifier = new DefaultCodeVerifier(codeGenerator, timeProvider);
        return verifier.isValidCode(secret, code);
    }
}
```

TOTP Configuration:

- **Algorithm:** SHA-1 (compatible with most authenticator apps)
- **Digits:** 6-digit codes (industry standard)
- **Period:** 30-second intervals (balance security and usability)
- **Window:** ± 1 time period for clock drift tolerance

⚙️ Configuration Management

JWT Configuration Properties

Externalized Configuration

```
app:
  jwt:
    secret: "mySuperSecretKeyThatIsAtLeast32CharactersLong123!"
    access-token-ttl: 15m      # Short-lived for security
    refresh-token-ttl: 30d    # Long-lived for user convenience
    issuer: "hrms-suite"      # Token issuer identification
  token:
    access-token-ttl: 15m
    refresh-token-ttl: 30d
    token-rotation-enabled: true # Security best practice
    max-refresh-tokens-per-user: 5 # Prevent token accumulation
```

Configuration Bean

```

@ConfigurationProperties(prefix = "app.jwt")
@Data
@Component
public class JwtConfigProperties {
    private String secret;
    private Duration accessTokenTtl = Duration.ofMinutes(15);
    private Duration refreshTokenTtl = Duration.ofDays(30);
    private String issuer = "hrms-suite";
}

```

Security Recommendations:

- **Production secret:** Environment variable or secret manager
- **Short access tokens:** 15 minutes for reduced exposure window
- **Secure token rotation:** Prevents refresh token reuse

Security Configuration

Comprehensive Security Setup

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(AbstractHttpConfigurer::disable) // Stateless API - JWT protected
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/api/v1/auth/login").permitAll()
                .requestMatchers("/api/v1/auth/refresh").permitAll()
                .requestMatchers("/api/v1/auth/password-reset/**").permitAll()
                .requestMatchers("/api/v1/auth/introspect").hasRole("SYSTEM")
                .requestMatchers("/api/v1/auth/revoke").hasAnyRole("ADMIN", "SYSTEM")
                .requestMatchers("/api/v1/users/**").authenticated()
                .requestMatchers("/api/v1/roles/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOriginPatterns(Arrays.asList("*")); // Configure appropriately
        configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        configuration.setAllowedHeaders(Arrays.asList("*"));
        configuration.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}

```

CORS Security Considerations:

- **Origin patterns:** Restrict to known domains in production
- **Credentials:** Allow cookies/auth headers
- **Methods:** Limit to required HTTP methods

Database Configuration

PostgreSQL Setup

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/hrms
    username: hrms_user
    password: ${DB_PASSWORD:defaultPassword}
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: validate          # Safety - no auto schema changes
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
  sql:
    init:
      mode: always                # Always run schema.sql on startup
```

Database Best Practices:

- **Schema validation:** Prevent accidental schema changes
- **SQL formatting:** Development debugging aid
- **Explicit initialization:** Controlled database setup

📄 Complete Data Flow

User Registration Flow

```
1. HTTP POST /api/v1/users
  ↓
2. UserAdminController.createUser()
  ↓ @PreAuthorize("hasRole('ROLE_ADMIN') or hasRole('ROLE_HR')")
  ↓
3. UserService.createUser()
  ↓ Validation: username/email uniqueness
  ↓ Password hashing with Argon2
  ↓ Database persistence
  ↓
4. Event: UserCreatedEvent published
  ↓
5. Response: UserDto with masked sensitive data
```

Authentication Flow

```
1. HTTP POST /api/v1/auth/login
  ↓
2. AuthController.login()
  ↓ Input validation (@Valid)
  ↓
3. AuthService.authenticate()
  ↓ User lookup (case-insensitive)
  ↓ Account status check (ACTIVE/DISABLED/LOCKED)
  ↓ Password verification (timing-attack resistant)
  ↓ MFA validation (if enabled)
  ↓ Failed login counter management
  ↓
4. TokenService token generation
  ↓ JWT access token (15min expiry)
  ↓ Refresh token (30 days, hashed storage)
  ↓
5. AuditService event logging
  ↓ Login success/failure records
  ↓
6. Response: AuthResponse with tokens and user info
```

Token Refresh Flow

```
1. HTTP POST /api/v1/auth/refresh
  ↓
2. AuthController.refreshToken()
  ↓
3. AuthService.refreshToken()
  ↓ Refresh token validation (hash lookup)
  ↓ Token expiration/revocation checks
  ↓ User status validation
  ↓ Token rotation (revoke old, issue new)
  ↓
4. AuditService token refresh logging
  ↓
5. Response: New access and refresh tokens
```

Role Assignment Flow

```
1. HTTP POST /api/v1/users/{id}/roles
  ↓ @PreAuthorize("hasRole('ROLE_ADMIN')")
  ↓
2. RoleService.assignRoleToUser()
  ↓ Role existence validation
  ↓ Duplicate assignment prevention
  ↓ UserRole entity creation
  ↓
3. Database: user_roles record with assigned_at
  ↓
4. Response: HTTP 204 No Content
```

🔒 Security Considerations

Authentication Security

Password Security

- **Argon2id:** Memory-hard algorithm resistant to GPU attacks
- **Salt integration:** Automatic salting prevents rainbow tables
- **Timing attack protection:** Constant-time comparison

Session Management

- **Short-lived access tokens:** 15-minute expiry limits exposure
- **Long-lived refresh tokens:** 30-day convenience with secure storage
- **Token rotation:** Prevents refresh token replay attacks
- **Comprehensive revocation:** Manual and automatic token invalidation

Authorization Security

Role-Based Access Control

- **Fine-grained permissions:** Method-level security annotations
- **Context-aware access:** Self-reference for personal data
- **Hierarchical roles:** Manager > Employee relationships

API Security

- **Input validation:** Bean Validation with custom constraints
- **SQL injection prevention:** JPA with parameterized queries
- **XSS protection:** JSON serialization with proper content types

Data Protection

Sensitive Data Handling

- **Password hashing:** Never stored in plaintext
- **MFA secret encryption:** Application-level encryption before storage
- **Token hashing:** Refresh tokens hashed in database
- **Audit log sanitization:** No sensitive data in logs

Database Security

- **UUID primary keys:** Unpredictable identifiers
- **Foreign key constraints:** Data integrity enforcement
- **Index optimization:** Performance without compromising security
- **Soft deletion:** Audit trail maintenance

Operational Security

Monitoring & Auditing

- **Comprehensive audit logs:** All authentication events recorded
- **Security event tracking:** Failed logins, token revocations
- **Compliance readiness:** GDPR, SOC2, ISO27001 support

Error Handling

- **Generic error messages:** Prevent information leakage
- **Structured error responses:** Consistent API error format
- **Security event alerts:** Suspicious activity detection

🔒 Conclusion

This comprehensive Module 1 implementation provides:

🔑 Core Authentication Features

- Username/password authentication
- JWT-based stateless sessions
- Multi-factor authentication (TOTP)
- Secure password reset flow

👤 Identity Management

- User lifecycle management (create, read, update, disable)
- Role-based access control (RBAC)
- Employee identity correlation
- System user provisioning

🔐 Security Foundation

- Cryptographic password storage

- Secure token management
- Comprehensive audit logging
- Rate limiting readiness

☒ Integration Ready

- Event-driven architecture
- RESTful API design
- Cross-module correlation
- External system integration points

☒ Production Ready

- Database schema with indexes
- Configuration management
- Error handling
- Security best practices

The module is designed to be **secure by default, highly scalable**, and **easily maintainable** while providing a solid foundation for the entire HRMS suite.